

one compression pipeline (§4) writes it · all three kernels read it — no decompression, no re-encode

shared compressed block store $\{U, V_K, V_V, a_k, a_v, R_K, R_V, k_{\min}, k_{\max}\}$

routed decode attention — backend-agnostic

MLX / Metal

Apple silicon · unified memory

MLXDKVWrapper →
patched Qwen2 attention

mx.matmul
 qV_K^T and U -expand

$\text{mx.fast.metal_kernel}$
tiled UV expansion

scaled_dot_product_attention
residuals + recency window

evaluated throughout §8

CUDA / Triton

NVIDIA discrete GPU

native_triton_sparse_
attn_decode()

@triton.autotune over
 $S_{\text{MAX}} \times \text{BLOCKS/CHUNK} \times \text{warps}$

_fused_sparse_decode_kernel
one fused launch

PyTorch vectorized fallback
DKV_TRITON_STRICT=1 → error

GPU validation pending

native C++ / GGML

NVIDIA discrete GPU

execute_cuda_attention()

dkv_full_decode_kernel
dkv_decode.cu · -DGGML_CUDA=ON

split-K accumulation
scratch $H_q \times S_{\text{split}} \times d$

DKV_CUDA_CHECK on every
malloc / memcpy / launch

GPU validation pending

attention output o — one interface, one numerical contract